

Relatorio Tecnico do Simulador de Trafego Urbano

Alan Mendes Vieira, Diogo Gomes Figueiredo, Antonio Pereira Da Luz Neto, Cicero Jesus Da Silva Gomes

Resumo—Este relatorio descreve, de forma resumida, os principais recursos utilizados no desenvolvimento do Simulador de Trafego Urbano e como eles foram aplicados no trabalho. O projeto foi implementado em C com uso de threads, mutexes e variaveis de condicao, tendo como foco a simulacao concorrente de veiculos em uma malha urbana. Sao apresentadas as principais decisoes de implementacao relacionadas ao mapa, as threads, aos mecanismos de sincronizacao, a ausencia de espera ocupada, a prioridade da ambulancia, a validacao e a estrategia adotada para reduzir o risco de deadlock.

I. INTRODUCAO

O Simulador de Trafego Urbano foi desenvolvido como trabalho pratico da disciplina de Sistemas Operacionais. O objetivo foi construir uma simulacao em que varios veiculos pudessem circular por uma malha urbana compartilhada, respeitando celulas ocupadas, cruzamentos, semaforos e o avanço do tempo da simulacao.

Para isso, o projeto utilizou a linguagem C, a biblioteca `pthread` em sistemas POSIX e uma camada de compatibilidade para Windows em alguns modulos. A simulacao foi organizada em torno de cinco elementos principais: mapa, celulas, vias, cruzamentos e veiculos. A coordenacao entre esses elementos foi feita por threads, mutexes e variaveis de condicao.

A demonstracao integrada executa uma matriz de 20 por 40 celulas, tres vias horizontais, quatro vias verticais e 12 cruzamentos. Foram utilizados 15 carros comuns e tres ambulancias, cada veiculo em sua propria thread, durante 60 ticks de um segundo. Os veiculos rapidos avancam a cada tick, os medios a cada dois ticks e os lentos a cada quatro ticks. Os sinais alternam a cada tres ticks, exceto quando uma travessia em andamento ou uma prioridade de ambulancia exige adiar a mudanca.

As principais fontes consideradas para este relatorio foram o plano de tarefas do projeto, o README e os arquivos de codigo-fonte presentes no diretorio `src/`.

II. MAPA

O mapa foi implementado pela estrutura `CityMap`. Ele representa a cidade como uma matriz de celulas com dimensoes fixas de 20 linhas por 40 colunas. Cada posicao da matriz e uma `Cell`, que pode estar livre ou ocupada por um veiculo.

Cada `Cell` armazena coordenadas, estado de ocupacao, simbolo exibido e ponteiro para o veiculo presente na posicao. Esse estado e protegido por mutex proprio, permitindo que a celula seja consultada e atualizada de forma segura pelas threads de veiculos e pelo renderizador.

As vias foram representadas pela estrutura `Road`. O mapa possui vias horizontais e verticais predefinidas. No projeto, foram utilizadas tres vias horizontais e quatro vias verticais,

formando doze cruzamentos. Cada via nao cria novas celulas; ela guarda ponteiros para as celulas reais da matriz do mapa. Essa decisao evita duplicacao de estado e garante que, quando um veiculo ocupa uma posicao, essa ocupacao seja visivel para qualquer via que passe pela mesma celula.

As rotas da demonstracao sao fixas e percorrem uma unica via. As vias de mao dupla foram representadas por uma unica faixa logica compartilhada, sem modelar duas faixas fisicas independentes.

Os cruzamentos foram representados pela estrutura `Intersection`. Cada cruzamento aponta para uma via horizontal e uma via vertical, alem de armazenar a direcao atualmente liberada pelo semaforo. Assim, o estado do cruzamento fica concentrado em um unico objeto, evitando a existencia de duas fontes diferentes para o mesmo sinal.

Uma decisao importante foi deixar o `CityMap` como dono das celulas, vias e cruzamentos. As outras estruturas apenas referenciam esses objetos. Isso tornou mais clara a ordem de criacao e destruicao dos recursos, reduzindo problemas como ponteiros invalidos, liberacao dupla de memoria e uso de celulas depois que o mapa ja foi destruido.

III. THREADS

O projeto utilizou threads para representar execucoes concorrentes dentro da simulacao. A principal thread auxiliar e a do relógio global, implementada no modulo `GlobalClock`. Ela incrementa o `global_tick` periodicamente e acorda as demais threads que estao esperando a passagem do tempo.

Os veiculos tambem foram modelados para executar em threads. Cada veiculo possui seu proprio fluxo de execucao, aguarda o tick global e tenta se mover conforme sua velocidade, rota e estado atual da via. Com isso, a simulacao reproduz disputas reais por recursos compartilhados, como celulas e cruzamentos.

Cada veiculo possui identificador, tipo, posicao, direcao, velocidade, rota e estado. A thread do veiculo chama `wait_next_tick()` e acumula os ticks exigidos por sua velocidade antes de tentar executar um unico movimento valido. Antes de avancar, a thread verifica limites do mapa, adjacencia, direcao, regra de mao unica, sinal do cruzamento e disponibilidade da celula de destino.

A ambulancia foi tratada como uma especializacao de veiculo. Ela reutiliza a mesma base de movimento, celulas, vias e sincronizacao, mas e renderizada com o simbolo A. Essa decisao evitou criar uma segunda logica independente de movimentacao e manteve as mesmas regras de seguranca usadas pelos veiculos comuns.

Ao se aproximar de um cruzamento, a ambulancia registra o pedido de prioridade e chama

`intersection_request_ambulance_priority()`. Se houver travessia conflitante em andamento, ela aguarda a condicao de liberacao. Quando a passagem e segura, o cruzamento guarda o sinal anterior, libera a direcao solicitada e acorda as threads dessa direcao. Ao sair, a ambulancia libera a prioridade e o cruzamento restaura a direcao anterior.

Tambem foi criada sincronizacao para a saida no terminal. Como diferentes threads podem gerar logs ou frames ASCII, o modulo `SimulationOutput` utiliza um mutex proprio para impedir que mensagens sejam impressas de forma intercalada.

IV. MECANISMOS DE SINCRONIZACAO

O principal mecanismo de sincronizacao utilizado foi o mutex. Cada celula do mapa possui um `pthread_mutex_t`. Quando um veiculo tenta ocupar uma celula, a funcao de ocupacao bloqueia o mutex daquela celula, verifica se ela esta livre, registra o veiculo e libera o mutex. Dessa forma, dois veiculos nao conseguem ocupar a mesma celula ao mesmo tempo.

Essa decisao foi melhor do que usar um unico mutex global para todo o mapa, pois permite maior paralelismo. Veiculos em regioes diferentes podem se mover sem bloquear toda a simulacao. Apenas veiculos que disputam a mesma celula precisam competir pelo mesmo mutex.

Nos cruzamentos, tambem foram usados mutexes e variaveis de condicao. Cada `Intersection` possui um mutex, uma condicao para a direcao horizontal e outra para a direcao vertical. Quando o sinal de uma direcao esta fechado, a thread do veiculo pode dormir aguardando a condicao correspondente. Quando o sinal muda, as threads daquela direcao sao acordadas.

O cruzamento tambem armazena a direcao verde, o estado de prioridade da ambulancia e o estado de travessia. A thread mantem o mutex do cruzamento desde a confirmacao da permissao ate a ocupacao efetiva da celula, impedindo que o sinal mude entre a verificacao e a entrada. Enquanto a celula do cruzamento contem um veiculo, a alternancia periodica do sinal e adiada. Na saida, a thread limpa o estado de travessia e acorda quem aguarda a liberacao.

O relógio global tambem usa mutex e variavel de condicao. As threads chamam `wait_next_tick()` para aguardar o proximo tick. Quando o relógio incrementa o valor global, ele executa um `broadcast`, acordando todas as threads que dependem do tempo da simulacao.

A transferencia entre origem e destino ocorre sob o mutex de estado do mapa e os mutexes das duas celulas envolvidas. Os mutexes das celulas sao sempre adquiridos por uma ordem global baseada em linha, coluna e endereco, e liberados na ordem inversa. Essa regra torna a troca atomica para as demais threads e para o renderizador.

V. AUSENCIA DE ESPERA OCUPADA

Uma preocupacao importante do projeto foi evitar espera ocupada. Espera ocupada ocorre quando uma thread fica em loop verificando repetidamente se uma condicao mudou, consumindo CPU sem realizar trabalho util.

No simulador, a passagem do tempo nao e aguardada por loops ativos. As threads usam variaveis de condicao para dormir ate que o relógio global avance. Assim, um veiculo que precisa esperar o proximo tick nao fica testando continuamente o valor de `global_tick`; ele fica bloqueado ate receber o sinal da thread do relógio.

O mesmo principio foi aplicado aos cruzamentos. Quando um veiculo encontra um sinal vermelho, ele nao fica consultando o semaforo em loop. A thread aguarda uma variavel de condicao associada ao cruzamento e so volta a executar quando a direcao correspondente e liberada.

Tambem nao ha espera ativa quando uma ambulancia precisa aguardar uma travessia conflitante. Nesse caso, a thread dorme na condicao do cruzamento e so volta a concorrer pela passagem quando o estado de travessia e liberado.

Essa decisao deixou a simulacao mais eficiente e mais alinhada com os conceitos de Sistemas Operacionais estudados na disciplina, pois usa bloqueio e notificacao em vez de desperdicar processamento com verificacoes repetitivas.

VI. ESTRATEGIA CONTRA DEADLOCK

A estrategia contra deadlock foi baseada em reduzir o tempo de posse dos locks, evitar esperas segurando recursos desnecessarios e definir contratos consistentes de aquisicao. No movimento dos veiculos, nenhuma thread espera por tick, sinal ou liberacao de travessia segurando mutex de celula. Quando o movimento envolve origem e destino, os pares de celulas seguem a ordem global de aquisicao por linha, coluna e endereco.

Outra decisao importante foi separar espera por sinal da ocupacao de celula. Quando um veiculo precisa aguardar um cruzamento, ele usa o mutex e a variavel de condicao do cruzamento, mas nao fica segurando o mutex de uma celula enquanto espera o semaforo abrir. Isso reduz o risco de uma thread impedir outras de avancarem enquanto esta bloqueada por um evento externo.

O mesmo cuidado foi aplicado a prioridade da ambulancia. A ambulancia pode solicitar prioridade no cruzamento, mas a ocupacao exclusiva da celula continua obrigatoria. Se existir travessia conflitante, a thread aguarda por variavel de condicao, sem reter mutexes de celulas. Quando a prioridade termina, o cruzamento restaura o sinal anterior e acorda as threads que podem prosseguir.

A prevencao de deadlock combina quatro decisoes principais: esperas por tick, sinal e liberacao usam variaveis de condicao; nenhuma thread espera por evento segurando mutex de celula; pares de celulas seguem uma ordem global de aquisicao; e mutexes de mapa, celulas e cruzamento possuem secoes criticas curtas.

O projeto tambem adotou uma ordem clara de encerramento. Ao finalizar a simulacao, a flag global `simulation_running` e alterada, as threads bloqueadas sao acordadas por `broadcast`, as threads sao aguardadas com `join` e somente depois os recursos do mapa, do relógio e da saida sao destruidos. Essa ordem evita que uma thread acorde e tente acessar uma estrutura que ja foi liberada.

Por fim, a simulacao manteve locks com escopo pequeno. Cada mutex protege apenas o recurso necessario: celula, cruzamento, relógio ou saida. Essa organizacao diminui a chance de espera circular entre threads, que e uma das condicoes classicas para ocorrencia de deadlock.

VII. VISUALIZACAO E VALIDACAO

O terminal e atualizado a cada tick. O simbolo C representa carro, A representa ambulancia, H representa sinal horizontal e V representa sinal vertical. O mapa permite observar movimento, bloqueios e mudancas de sinal. Os pedidos, concessoes e liberacoes de prioridade da ambulancia sao registrados em `bin/simulation.log`, enquanto os frames permanecem isolados no terminal.

O alvo `make test` verifica criacao e destruicao do mapa, prioridade da ambulancia, restauracao do ciclo, adiamento do sinal durante travessia, movimento sem ocupacao duplicada, bloqueio por celula ocupada, mao unica, mao dupla, limites do mapa, arquivo de eventos e formato do renderizador. A validacao final tambem executa os 60 ticks e exige encerramento normal.

VIII. INTEGRANTES E RESPONSABILIDADES

Alan Mendes Vieira ficou responsavel por mapa, celulas, vias e integracao. Diogo Gomes Figueiredo ficou responsavel por veiculos, rotas e threads. Antonio Pereira da Luz Neto ficou responsavel por relógio, sinais e encerramento. Cicero Jesus da Silva Gomes ficou responsavel pelo renderizador e pela ambulancia.

IX. CONCLUSAO

O trabalho utilizou conceitos centrais de Sistemas Operacionais em uma aplicacao pratica: threads para concorrencia, mutexes para exclusao mutua, variaveis de condicao para bloqueio eficiente e uma estrategia de encerramento para evitar threads presas. O mapa foi modelado como uma matriz compartilhada de celulas, e os veiculos interagiram com essa matriz por meio de regras de ocupacao protegidas por mutex.

As principais decisoes de implementacao buscaram manter o estado compartilhado consistente, evitar espera ocupada e reduzir riscos de deadlock. Dessa forma, o projeto serviu como uma aplicacao direta dos conteudos de sincronizacao, concorrencia e gerenciamento de recursos estudados na disciplina.

REFERÊNCIAS

- [1] Equipe do projeto, “Codigo-fonte do Simulador de Trafego Urbano”, 2026.